RAPTOR: Robust and Perception-aware Trajectory Replanning for Quadrotor Fast Flight

Boyu Zhou, Jie Pan, Fei Gao, and Shaojie Shen

arXiv:2007.03465v1 [cs.RO] 6 Jul 2020

***Abstract*—Recent advances in trajectory replanning have enabled quadrotor to navigate autonomously in unknown environments. However, high-speed navigation still remains a significant challenge. Given very limited time, existing methods have no strong guarantee on the feasibility or quality of the solutions. Moreover, most methods do not consider environment perception, which is the key bottleneck to fast flight. In this paper, we present RAPTOR, a robust and perception-aware replanning framework to support fast and safe flight, which addresses these issues systematically. A path-guided optimization (PGO) approach that incorporates multiple topological paths is devised, to ensure finding feasible and high-quality trajectories in very limited time. We also introduce a perception-aware planning strategy to actively observe and avoid unknown obstacles. A risk-aware trajectory refinement ensures that unknown obstacles which may endanger the quadrotor can be observed earlier and avoid in time. The motion of yaw angle is planned to actively explore the surrounding space that is relevant for safe navigation. The proposed methods are tested extensively through benchmark comparisons and challenging indoor and outdoor aggressive flights. We will release our implementation as an open-source package[1] for the community.**

***Index Terms*—Aerial systems: perception and autonomy, collision avoidance, motion and path planning, trajectory planning.**

I. Introduction

In recent years, progresses on different aspects of unmanned aerial vehicles (UAVs), especially quadrotor autonomy have been achieved and promote autonomous navigation. Nonetheless, high-speed flight in unknown and highly cluttered environments still remains one of the biggest challenges toward full autonomy. To achieve fast flight, trajectory replanning is of vital importance to cope with previously unknown obstacles, guaranteeing smooth and safe navigation.

Although trajectory replanning has been investigated actively, most presented methods only apply to flights at a moderate speed. Several issues greatly hinder their usage in high-speed scenarios. **(a)** Flying in unknown environments at a high speed, the quadrotor should replan new trajectories to avoid unexpected obstacles in considerably short time, otherwise it crashes. However, most methods do not guarantee to find feasible trajectories given very limited time. **(b)** Current methods typically find a locally optimal trajectory confined within a topologically equivalent class, which does not necessarily contains a satisfactory solution for smooth and safe navigation, especially in fast flight. **(c)** Existing methods are unaware of environment perception, which can be fatal as the flight speed and obstacle density get high. Paying no attention to perception, the planned motions may lead to restricted visibility to the environments, which would in turn result in deficient information of the surrounding space necessary for safe navigation. The consequence of not considering perception in replanning can be better illustrated by Fig.1. To minimize the energy consumption, a trajectory near the wall is generated, along which the visibility toward the unknown space behind the corner is very limited. As a result the obstacle right behind the corner is invisible until the quadrotor turns right and gets very close, which 'surprises' or even crashes the quadrotor. Instead of avoiding what are observed passively, actively observing and avoiding possible dangers is critical for safe high-speed flight.

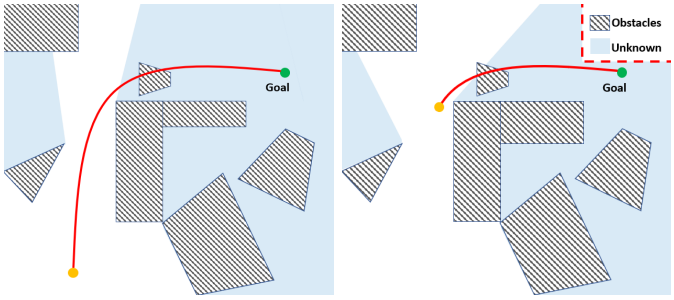


Fig. 1. An example of planning without perception awareness. The quadrotor flies near the wall, where it has poor visibility to the space behind the corner. In consequence, an obstacle is not revealed until the quadrotor gets very close.



Fig. 2. Composite image of a fast flight experiment in a cluttered unknown environment. Video is available at: https://youtu.be/6wrh4G1-cQ4.

Boyu Zhou, Jie Pan and Shaojie Shen are with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong, China. {bzhouai, jpanai, eeshaojie}@ust.hk
Fei Gao is with the State Key Laboratory of Industrial Control and Technology, Zhejiang University, Hangzhou 310007, China. fgaoaa@zju.edu.cn

[1]To be released at github.com/HKUST-Aerial-Robotics/Fast-Planner

In this paper, we propose a **R**obust **A**nd **P**erception-aware **T**raject**O**ry **R**eplanning framework called **RAPTOR** to address these issues systematically. To ensure obtaining feasible trajectories within limited time, we present a path-guided gradient-based optimization method, which utilizes geometric guiding paths to eliminate infeasible local minima and guarantee the success of replanning. Also, to further improve the optimality of the replanning, we introduce an online topological path planning to extract a comprehensive set of paths that capture the structure of the environment. With the guidance of several distinctive paths, multiple trajectories are optimized in parallel, leading to a more thorough exploration of the solution space.

The above mentioned method that address issues **(a)** and **(b)** were first proposed in our previous work [1]. However, it adopts the optimistic assumption and lack the awareness of environment perception, which restricts its capability in higher speed and more complex environments. To bridge this gap, we extend it with a perception-aware planning strategy to enable faster and safer flight from two aspects. Firstly, a risk-aware trajectory refinement approach is developed to incorporate with the optimistic planner. It identifies unknown regions along the optimistic trajectories that are potentially dangerous to the quadrotor. Visibility toward such regions along with safe reaction distance are enforced explicitly, ensuring that obstacles in unmapped areas become visible earlier and are avoidable by the quadrotor. Secondly, we incorporate the yaw angle of the quadrotor into a two-step motion planning framework. A optimal sequence of yaw angles that maximizes information gain and smoothness is searched in the discrete state space, which is further smoothed through optimization. The planned motions of yaw angle enable the quadrotor whose field-of-view (FOV) is limited to actively explore the unknown space to gain more relevant knowledge for the future flight.

We conduct systematic evaluations on both the proposed perception-aware planning strategy and the whole planning system, through benchmark comparisons and challenging real-world experiments. For the former, it is able to support fast and safe flight in challenging scenarios where traditional method fail to ensure safety. For the later, our planner outperform state-of-the-art methods in several aspects in fast flight tasks. Extensive indoor and outdoor flight tests in complex environments also validates our planning system. The contributions of this paper are summarized as follows:

1) A topological paths-guided gradient-based replanning approach, that is capable of generating high-quality trajectories in limited time.

2) A risk-aware trajectory refinement approach, which enforces visibility and safe reaction distance to unknown obstacles. It improves the predictability and safety of fast flights.

3) A two-step yaw angle planning method, to actively explore the unknown environments and gather useful information for the flight.

4) Extensive simulation and real-word tests that validate the proposed method. The source code of our system will be released as an open-source package.

II. Related Work

A. Quadrotor Trajectory Planning

Trajectory planning for quadrotor has been widely investigated. Existing methods can be categorized into hard-constrained methods and gradient-based optimization methods. Hard-constrained methods are pioneered by minimum-snap trajectory [2], in which piecewise polynomial trajectories are generated through quadratic programming(QP). [3] presented a closed-form solution to minimum snap trajectories. [4]–[8] generate trajectories in a two-step pipeline. Safe regions around initial paths are extracted as convex flight corridors, within which QP is solved to generate smooth and safe trajectories. Among these methods, poorly chosen time allocation of piecewise polynomials usually lead to unsatisfying results. To this end, fast marching [5] and kinodynamic search [8, 9] are utilized to search for initial paths with more reasonable time allocation. [5] also proposed to represent trajectories as piecewise Bézier curves so that safety and dynamical feasibility are guaranteed. [10] adopted a mixed integer QP formulation to find a more reasonable time allocation of the trajectory.

Another category is the gradient-based trajectory optimization (GTO) methods, which typically formulate trajectory generation as non-linear optimization problems trading off smoothness, safety and dynamic feasibility. Recently works [1, 11]–[15] revealed that they are particularly effective for local replanning, which is a key component for high-speed flight in unknown environments. [16] proposed to optimize discrete-time trajectory through covariant gradient descent, reviving the community's interest in such methods. [17] presented a similar formulation, but solves the problem by sampling neighboring candidates iteratively. The stochastic sampling strategy partially overcomes the local minima issue but is computationally intensive. [11] extended it to continuous-time quadrotor trajectories and also adopted random optimization restarts to slightly relieve the typical local minima issue of such methods. In [12, 13], the optimization is combined with an informed sampling-based path searching to improve the success rate. [14] proposed to parameterize trajectories as uniform B-splines, showing the usefulness of continuity and locality properties of B-spline for replanning. However, due to insufficient success rate and efficiency, [11, 12, 14] only apply to flight at a moderate speed. To this end, [15] further exploited B-spline to improve the efficiency and robustness. GTO methods are preferable for replanning due to high efficiency. However, their local minima issue may lead to undesired solutions. Our method lies in this category, and we resolve local minima by introducing topologically distinctive paths to guide optimization in parallel.

B. Topological Path Planning

There are works utilizing the idea of topologically distinct paths for planning, in which paths belonging to different homotopy (homology) [18]–[22] or visibility deformation [23] classes are sought. [18] constructs a variant of probabilistic roadmap (PRM) to capture homotopy classes, in which path

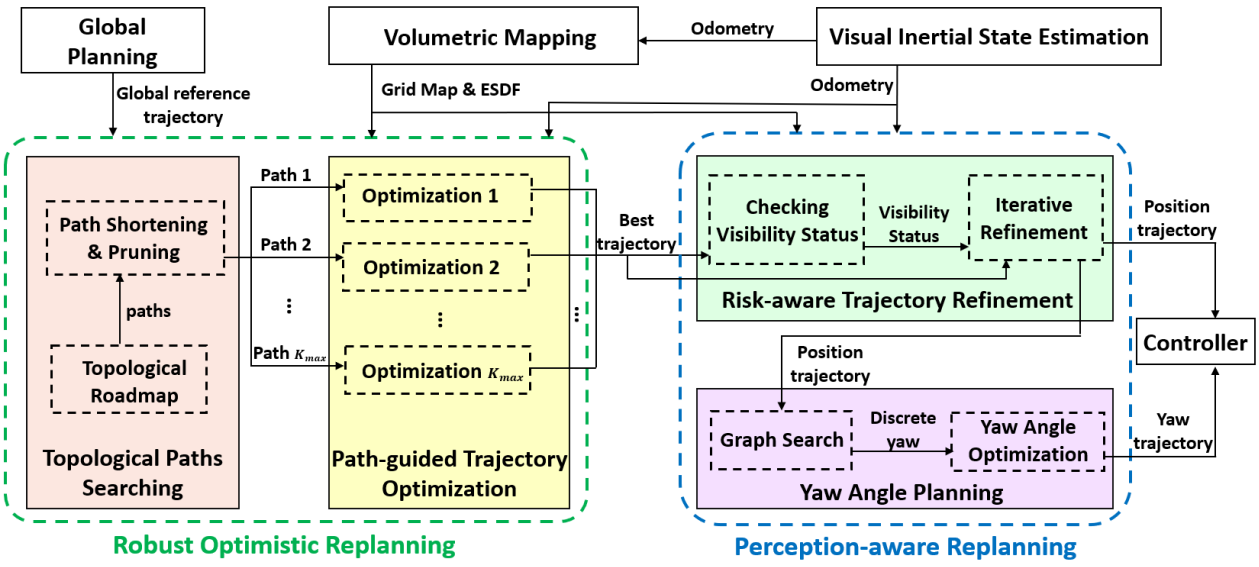


Fig. 3. An overview of our replanning system. It takes information from the global planning, mapping and state estimation and replans trajectory in two steps.

searching and redundant path filtering are conducted simultaneously. In contrast, [19, 20] firstly creates a PRM or Voronoi diagram, after which a homology equivalence relation based on complex analysis [21] is adopted to filter out redundant paths. These methods only apply to 2D scenarios. To seek for 3D homology classes, [22] exploit the theory of electromagnetism and propose a 3D homology equivalence relation. However, it requires occupied space to be decomposed into "genus 1" obstacles, which is usually impractical. Besides, capturing only homotopy classes in 3D space is insufficient to encode the set of useful paths, as indicated in [23], since 3D homotopic paths may be too hard to deform into each other. To this end, [23] leverages a visibility deformation roadmap to search for a richer set of useful paths. [24, 25] convert maps built from SLAM systems into sparse graphs representing the topological structure of the environments. [23]–[25] focus on global offline planning and is too time-consuming for online usage. Our topological path searching is conceptually closest to [23], but with a reinvented algorithm for real-time performance.

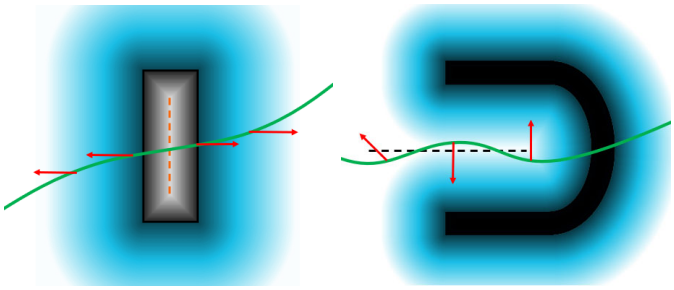
C. Navigation in Unknown Environments

To deal with unknown environments in navigation, different strategies have been used. Many methods adopt the optimistic assumption [7, 14, 15, 26], which treats the unknown space as collision-free. This strategy improves the chance of reaching goals, but may not guarantee safety. On the contrary, some other methods regard unknown space as unsafe and only allow motions within the known-free space [27] or sensor FOV [28, 29]. In [30] the sensor FOV constraint is partially relaxed by choosing safe motion primitives generated in the past. Although these restrictions ensure safety, they lead to conservative motion. Recently [10] proposed a strategy that plans in both the known-free and unknown space. Instead of being over optimistic about the unknown space, it always maintains back-up trajectories to ensure safety.

The limitation of the above mentioned strategies is the lack of environment perception awareness, which is of significant necessity in fast flight. Although much attention has been paid to planning with the awareness of localization [31]–[33] and target tracking [34, 35], less emphasis is put on environment perception. [36] proposed a learned heuristic function to guide the path searching into areas with greater visibility to unknown space, but it may not generalize well to complex 3D environments. [37] showed an integrated mapping and planning framework for active perception. The planner iteratively simulates future measurements after executing specific motions, predicts uncertainty of the map, and minimizes the replanning risk. Its main drawback is the prohibited runtime for online usage. In [38], a local planner is coupled with local exploration to safely navigate a cluttered environment. However, it conservatively selects intermediate goals within known-free space, which restricts the flight speed. In this paper, we present a perception-aware strategy to ensures that unknown dangers can be discovered and avoided early. It guarantees safety and does not lead to conservative behaviors.

III. System Overview

The proposed replanning system is illustrated in Fig.3. It takes the outputs of the global planning, dense mapping and state estimation modules, and deforms the global reference trajectory locally to avoid previously unknown obstacles. The replanning works in two steps. Firstly, the robust optimistic replanning generates multiple locally optimal trajectories in parallel through the path-guided optimization (Sect.IV). The optimization is guided by topologically distinctive paths extracted and carefully selected from the topological path searching, which will be detailed in Sect.V. Optimistic assumption is adopted in this step. Secondly, the perception-aware planning



(a) (b)

Fig. 4. Typical examples of optimization failure, where the trajectories pass through the 'I' shape and 'U' shape obstacles. (a) Adjacent parts of the trajectory are pushed in opposing directions since it crosses the "valley" of the ESDF (denoted by dashed lines). Negative distance is in gray color, red arrows denote gradients of ESDF. (b) The trajectory crosses the "ridge" .

strategy is utilized. The best trajectory among the locally optimal ones is further polished by a risk-aware trajectory refinement, in which its safety and visibility to the unknown and dangerous space is improved, as presented in Sect.VI. Based on the refined trajectory, the yaw angle is planned to actively explore the unknown environments (Sect.VII). The global planning, mapping, estimation and controller are introduced briefly in Sect.VIII-A.

IV. Path-Guided Trajectory Optimization

As showed in Sect.II-A, GTO methods are effective for replanning, but suffer from local minima. To further improve the robustness of replanning and ensure flight safety, we present PGO, which utilizes a geometric guiding path in the optimization to guarantee its success.

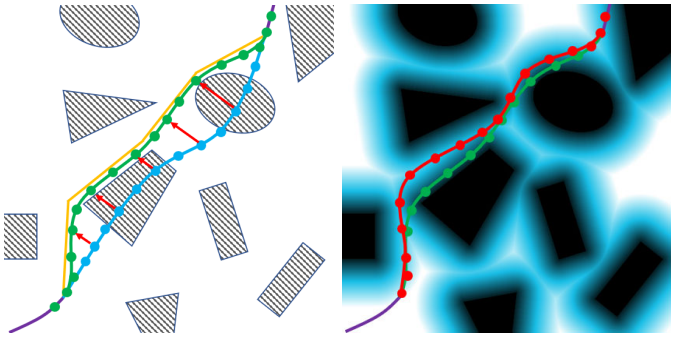
A. Optimization Failure Analysis

Previous work [39] showed that failure of GTO is relevant to unfavorable initialization, i.e., initial paths that pass through obstacles in certain ways usually get stuck. Underlying reason for this phenomenon is illustrated in Fig.4. Typical GTO methods incorporate the gradients of a Euclidean signed distance field (ESDF) in a collision cost to push the trajectory out of obstacles. Yet there are some "valleys" or "ridges" in the ESDF, around which the gradients differ greatly. Consequently, if a trajectory is in collision and crosses such regions, the gradients of ESDF will change abruptly at some points. This can make gradients of the collision cost push different parts of the trajectory in opposing directions and fail the optimization.

Normally, such "valleys" and "ridges", which corresponds to the space that has an identical distance to the surfaces of nearby obstacles, are difficult to avoid, especially in complex environments. Therefore, optimization depending solely on the ESDF fails inevitably at times. To solve the problem, it is essential to introduce extra information that can produce an objective function whose gradients consistently deform the trajectory to the free space.

B. Problem Formulation

We propose PGO built upon our previous work [15] that represents trajectories as B-splines for more efficient cost evaluation. For a trajectory segment in collision, we reparameterize it as a p_b degree uniform B-spline with control points $\{\mathbf{q}_0, \mathbf{q}_1, \ldots, \mathbf{q}_N\}$ and knot span Δt.



(a) (b)

Fig. 5. The two-phases PGO approach for trajectory replanning. (a) A geometric guiding path (yellow) attracts the original B-spline trajectory (blue) into the free space. (b) The warmup trajectory is further optimized in the ESDF to find a locally optimal trajectory (red).

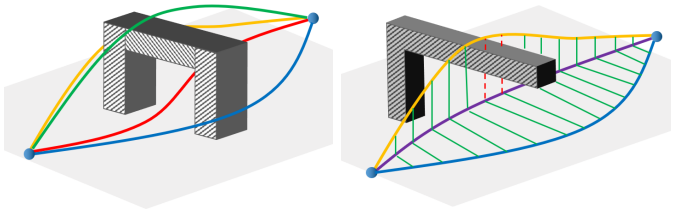
PGO consists of two different phases. The first phase generates an intermediate **warmup trajectory**. As concluded above, external information should be included to effectually deform the trajectory, since solely applying the ESDF could be futile. We employ a geometric guiding path to attract the initial trajectory to the free space (depicted in Fig. 5(a)) since collision-free paths are readily available from standard methods like A* and RRT*. In our work, the paths are provided by the sampling-based topological path searching (Sect. V). The first-phase objective function is:

$$f_{p1} = \lambda_s f_s + \lambda_g f_g = \tag{1}$$
$$\lambda_s \sum_{i=p_b-1}^{N-p_b+1} \|\mathbf{q}_{i+1} - 2\mathbf{q}_i + \mathbf{q}_{i-1}\|^2 + \lambda_g \sum_{i=p_b}^{N-p_b} \|\mathbf{q}_i - \mathbf{g}_i\|^2$$

where f_s is the smoothness term, while f_g penalize the distance between the guiding path and the B-spline trajectory. As in [15], f_s is designed as a elastic band cost function[2] that simulates the elastic forces of a sequence of springs. To simplify the design of f_g, we utilize the property that the shape of a B-spline is finely controlled by its control points. Each control point $\mathbf{q}_i$ is assigned with an associated point $\mathbf{g}_i$ on the guiding path, which is uniformly sampled along the guiding path. Then f_g is defined as the sum of the squared Euclidean distance between these point pairs.

Notably, minimizing f_{p1} yields an unconstrained quadratic programming problem, so its optimal solution can be obtained in closed form. It outputs a smooth trajectory in the vicinity of the guiding path. Since the path is already collision-free, usually the warmup trajectory is also so. Even though it is not completely collision-free, its major part will be attracted to the free space. At this stage, the gradients of ESDF along the trajectory vary smoothly, and the gradients of the objective function push the trajectory to the free space in consistent

[2]Only the subset of control points $\{\mathbf{q}_{p_b}, \mathbf{q}_{p_b+1}, \cdots, \mathbf{q}_{N-p_b}\}$ is optimized due to the boundary state constraints of the trajectory. $\mathbf{q}_{p_b-2}$, $\mathbf{q}_{p_b-1}$, $\mathbf{q}_{N-p_b+1}$ and $\mathbf{q}_{N-p_b+2}$ are needed to evaluate the smoothness.



(a) The green, blue and yellow trajectories are equivalent under the definition of homotopy, but represent substantially different motions.

(b) An illustration of UVD. The purple trajectory is distinctive to the yellow one, but is equivalent to the blue one.

Fig. 6. Topology equivalence relation.

directions. Hence, standard GTO methods can be utilized to improve the trajectory.

In the second phase, we adopt our previous B-spline optimization method [15] to further refine the warmup trajectory into a smooth, safe, and dynamically feasible one, whose objective function is:

$$f_{p2} = \lambda_s f_s + \lambda_c f_c + \lambda_d \left(f_v + f_a\right) = \tag{2}$$
$$\lambda_s \sum_{i=p_b-1}^{N-p_b+1} \left\|\mathbf{q}_{i+1} - 2\mathbf{q}_i + \mathbf{q}_{i-1}\right\|^2 + \lambda_c \sum_{i=p_b}^{N-p_b} \mathcal{F}(d(\mathbf{q}_i), d_{\min})$$
$$+ \lambda_d \sum_{\substack{\mu\in \\ \{x,y,z\}}} \left\{ \sum_{i=p_b-1}^{N-p_b} \mathcal{F}(v_{\max}^2, \dot{q}_{i,\mu}^2) + \sum_{i=p_b-2}^{N-p_b} \mathcal{F}(a_{\max}^2, \ddot{q}_{i,\mu}^2) \right\}$$

Here $\mathcal{F}()$ is a penalty function for general variables:

$$\mathcal{F}(x,y) = \begin{cases} (x-y)^2 & x \le y \\ 0 & x > y \end{cases} \tag{3}$$

f_c is the collision cost that grows rapidly if the trajectory gets closer than $d_{\min}$ to obstacles, where $d(\mathbf{q})$ is the distance of point $\mathbf{q}$ in the ESDF. f_v and f_a penalize infeasible velocity and acceleration, in which $\dot{\mathbf{q}}_i = [\dot{q}_{i,x}, \dot{q}_{i,y}, \dot{q}_{i,z}]^{\mathrm{T}}$ and $\ddot{\mathbf{q}}_i = [\ddot{q}_{i,x}, \ddot{q}_{i,y}, \ddot{q}_{i,z}]^{\mathrm{T}}$ are the control points of velocity and acceleration. They can be evaluated by:

$$\dot{\mathbf{q}}_i = \frac{\mathbf{q}_{i+1} - \mathbf{q}_i}{\Delta t}, \quad \ddot{\mathbf{q}}_i = \frac{\mathbf{q}_{i+2} - 2\mathbf{q}_{i+1} + \mathbf{q}_i}{\Delta t^2} \tag{4}$$

$v_{\max}$ and $a_{\max}$ are single-axis maximum velocity and acceleration. The formulations of f_c, f_v, and f_a are based on the convex hull property of B-spline, thanks to which it suffices to constrain the control points of the B-spline to ensure safety and dynamic feasibility. For brevity, we refer the readers to [15] for more details.

Although PGO has one more step of optimization compared with previous methods, it can generate better trajectories within shorter time. The first-phase takes only negligible time, but generate a warmup trajectory that is easier to be further refined, which improve the overall efficiency.

V. Topological Path Searching

Given a geometric guiding path, our PGO method can obtain a locally optimal trajectory. However, this trajectory is restricted within a topologically equivalent class and not necessarily satisfactory, even with the guidance of the shortest path, as seen in Fig. 8(e) and 8(f). Actually, it is difficult to determine the best geometric path, since the paths do not contain high order information (velocity, acceleration, etc.), and can not completely reflect the true motion. Searching a kinodynamic path [40, 41] may suffice, but it takes excessive time to obtain a promising path with boundary state constraints at the start and end of the replanned trajectory.

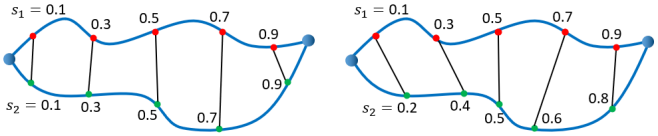


Fig. 7. Comparison between UVD (left) and VD (right). Each red point on one path is transformed to a green point on the other path. Any two associated points correspond to the same parameter s in UVD, but not in VD.

For a better solution, a variety of guiding paths are required. We propose a sampling-based topological path searching to find a collection of distinctive paths. Although methods [18, 23]–[25] are for this problem, none of them runs in real-time in complex 3D environments. We redesign the algorithm carefully to solve this challenging problem in real time.

A. Topology Equivalence Relation

Although the concept of homotopy is widely used, it captures insufficient useful trajectories in 3D environments, as shown in Fig. 6(a). [23] proposes a more useful relation in 3D space named visibility deformation (VD), but it is computationally expensive for equivalence checking. Based on VD, we define *uniform visibility deformation* (UVD), which also captures abundant useful trajectories, and is more efficient for equivalence checking.

Definition 1. Two trajectories $\tau_1(s), \tau_2(s)$ parameterized by $s \in [0,1]$ and satisfying $\tau_1(0) = \tau_2(0), \tau_1(1) = \tau_2(1)$, belong to the same *uniform visibility deformation* class, if for all s, line $\overline{\tau_1(s)\tau_2(s)}$ is collision-free.

Fig. 6(b) gives an example of three trajectories belonging to two UVD classes. The relation between VD and UVD is depicted in Fig. 7. Both of them define a continuous map between two paths $\tau_1(s)$ and $\tau_2(s)$, in which a point on $\tau_1(s)$ is transformed to a point on $\tau_2(s)$ through a straight-line. The major difference is that for UVD, point $\tau_1(s_1)$ is mapped to $\tau_2(s_2)$ where $s_1 = s_2$, while for VD s_1 does not necessarily equals s_2. In concept, UVD is less general and characterizes subsets of VD classes. Practically, it captures slightly more classes of distinct paths than VD, but is far less expensive [3] for equivalence checking.

To test UVD relation, one can uniformly discretize $s \in [0,1]$ to $s_i = i/K, i = 0, 1, ..., K$ and check collision for lines $\overline{\tau_1(s_i)\tau_2(s_i)}$. For the piece-wise straight line paths (as in Alg. 1, **Equivalent**()), we simply parameterize it uniformly, so that for any s except $\tau(s)$ is the join points of two straight lines, $\left\|\frac{d\tau(s)}{ds}\right\| = const.$

[3]To test VD relation, one should compute a visibility diagram and do path searching within it [23], which has higher complexity than testing UVD.

Algorithm 1: Topological Roadmap

```
1  Initialize()
2  AddGuard(G, s), AddGuard(G, g)
3  while t ≤ t_max ∧ N_sample ≤ N_max do
4      p_s ← Sample()
5      g_vis ← VisibleGuards(G, p_s)
6      if g_vis.size() == 0 then
7          AddGuard(G, p_s)
8      if g_vis.size() == 2 then
9          path_1 ← Path(g_vis[0], p_s, g_vis[1])
10         distinct ← True
11         N_s ← SharedNeighbors(G, g_vis[0], g_vis[1])
12         for each n_s ∈ N_s do
13             path_2 ← Path(g_vis[0], n_s, g_vis[1])
14             if Equivalent(path_1, path_2) then
15                 distinct ← False
16                 if Len(path_1) < Len(path_2) then
17                     Replace(G, p_s, n_s)
18                 break
19         if distinct then
20             AddConnector(G, p_s, g_vis[0], g_vis[1])
```

B. Topological Roadmap

Alg.1 is used to construct a UVD roadmap $\mathcal{G}$ capturing an abundant set of paths from different UVD classes. Unlike standard PRM containing many redundant loops, our method generates a more compact roadmap where each UVD class contains just one or a few paths (displayed in Fig.8(a)-8(c)).

We introduce two different kinds of graph nodes, namely *guard* and *connector*, similar to the Visibility-PRM [42]. The guards are responsible for exploring different part of the free space, and any two guards g_1 and g_2 are not *visible* to each other (line $\overline{g_1 g_2}$ is in collision). Before the main loop, two guards are created at the start point s and end point g. Every time a sampled point is invisible to all other guards, a new guard is created at this point (Line 6-7). To form paths of the roadmap, connectors are used to connect different guards (Line 7-19). When a sampled point is visible to exactly two guards, a new connector is created, either to connect the guards to form a topologically distinct connection (Line 19-20), or to replace an existing connector to make a shorter path (Line 16-17). Limits of time (t_{max}) or sampling number (N_{max}) are set to terminate the loop.

With the UVD roadmap, a depth-first search augmented by a visited node list is applied to search for the paths between s and g, similar to [20].

C. Path Shortening & Pruning

As shown in Fig. 8(d), some paths obtained from Alg. 1 may be detoured. Such paths are unfavorable, since in the first phase of PGO it can deform a trajectory excessively and make it unsmooth. Hence, Alg. 2 find a topologically equivalent shortcut path P_s for each P_r found by the depth-first search (illustrated in Fig. 9). The algorithm uniformly **Discretize**s P_r to a set of points P_d. In each iteration, if a point p_d in P_d is invisible from the last point in P_s (Line 3, 4), the center of the first occupied voxel blocking the view of P_s.**back**() is found (Line 5). This point is then pushed away from obstacles in the direction orthogonal to l_d and coplanar to both l_d and the ESDF gradient at p_b (Line 6), after which it is appended to P_s (Line 7). The process continues until the last point is reached.

Although in Alg. 1, redundant connection between two guards are avoided, there may exist a small number of redundant paths between s and g (Fig. 8(d)). To completely exclude repeated ones, we check the equivalence between any two paths and only preserve topologically distinct ones. Also note that the number of distinctive paths grows exponentially with the number of obstacles. In case of complex environments, it is computationally intractable to use all paths to guide parallel

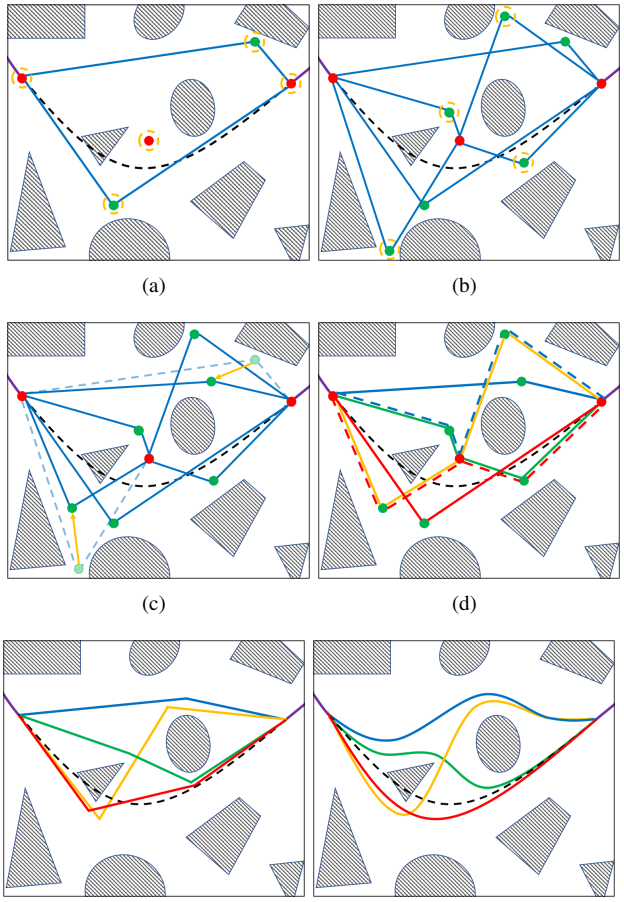


Fig. 8. Topological path searching and parallel trajectory optimization. (a): Two *connectors* (green) are sampled to connect the two initial *guards* (red) at the start and end points, after which a new *guard* is added to occupy a region not covered by others. (b): More *connectors* are created to connect the *guards*. (c): New *connectors* replace the old ones, making some connections shorter. (d): Paths are extracted from the roadmap. Equivalent paths are of the same color, pruned paths are showed as dashed line segments. (e): Extracted paths are shortened by Alg.2. (d): The shortened paths guide PGOs in parallel to generate several locally optimal trajectories. Note that the red trajectory is the smoothest since it takes less turns, while the associated guiding path is longer than others.

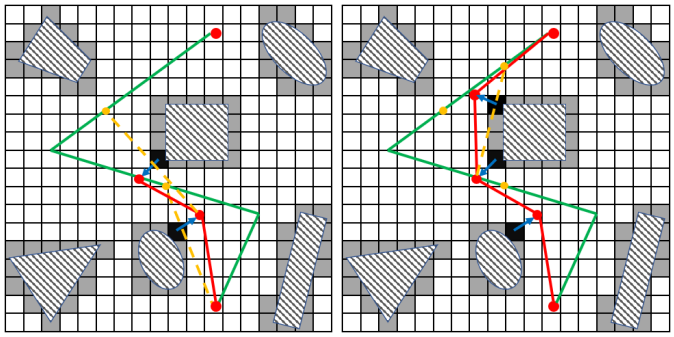


Fig. 9. A detoured and long path (green line) is shortened. The yellow discretized point are not visible to the last waypoint of the shortened path (red). The center points of the associated blocking voxel (black) are pushed away and appended as new waypoints.

Algorithm 2: Finding a topologically equivalent shortcut path P_s for P_r.

```
1 P_d ← Discretize(P_r), P_s ← {P_d.front()}
2 for each p_d ∈ P_d do
3     l_d ← Line(P_s.back(), p_d)
4     if ¬ LineVisib(l_d) then
5         p_b ← BlockPoint(l_d)
6         p_o ← PushAwayObs(p_b, l_d)
7         P_s.push_back(p_o)
8 P_s.push_back(P_d.back())
```

optimization. For this reason, we only select the first K_{max} shortest paths. Paths more than r_{max} times longer than the shortest one are also pruned away. Such strategies bound the complexity and will not miss the potentially optimal solution, because a very long path is very unlikely to result in the optimal trajectory.

VI. Risk-aware Trajectory Refinement

Our trajectory refinement takes the best trajectory from the parallel PGO $\mathbf{p}_i(t)$ as input, modifies it in its vicinity and outputs the refined trajectory $\mathbf{p}_r(t)$ (detailed in Alg.3). It starts by checking the visibility status of $\mathbf{p}_i(t)$, after which the visibility to relevant unknown space and the safe reaction distance are enforced in the iterative refinement.

A. Checking Visibility Status

The visibility status is encoded by several variables: $t_f, \mathbf{p}_f$, $t_c, \mathbf{p}_c, \mathbf{v}_c$, which are important information about the unknown space passed through by $\mathbf{p}_i(t)$. Some of the involved variables are illustrated in Fig.10.

1) Frontier Intersecting Point: As the unknown environment is only partially observed, at some time t_f the trajectory $\mathbf{p}_i(t)$ exits the known-free space and enters the unknown space. The position $\mathbf{p}_f = \mathbf{p}_i(t_f)$, should be prioritized for observation due to three reasons: First, it is highly relevant for the future flight, because it belongs to a promising trajectory going toward the goal. Second, it may be dangerous to the flight. In

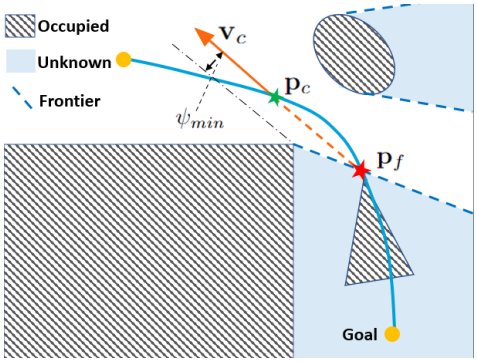


Fig. 10. An illustration of checking the visibility status of the inputted trajectory.

the worst case an unknown obstacle can be right adjacent to it. Third, it will be reached earlier compared to other unknown position along $\mathbf{p}_i(t)$. Therefore in **FrontierIntersection**() we search along $\mathbf{p}_i(t)$ with a discrete time step, recording the first unknown point and the corresponding time.

2) Visibility Metric: During the flight, it is preferred that $\mathbf{p}_f$ becomes visible to some preceding positions on $\mathbf{p}_i(t)$. Quantitatively, we want some visibility level ψ of $\mathbf{p}_f$ to be not less than an expected level ψ_{min}, so that not only $\mathbf{p}_f$ is visible but also the visibility level is tolerant to external disturbance. To gauge how reliable p_f is visible, we adopt the metric used by [35, 43], defining the visibility level from a position $\mathbf{p} \in \mathbb{R}^3$ to $\mathbf{p}_f$ as:

$$\psi(\mathbf{p}, \mathbf{p}_f) = \min_{\mathbf{q} \in l(\mathbf{p}, \mathbf{p}_f)} d(\mathbf{q}) \tag{5}$$

which represents the the smallest Euclidean signed distance between the line segment $l(\mathbf{p}, \mathbf{p}_f)$ and obstacles. The evaluation of Equ.5 requires traversing $l(\mathbf{p}, \mathbf{p}_f)$ and checking the Euclidean signed distance for each point. Fortunately, an ESDF derived from the occupancy grid map is maintained by our mapping module for supporting the trajectory optimization (Sect.IV), so minimal distance can be queried in constant time.

3) Critical View Direction: We are interested in whether $\mathbf{p}_f$ can be viewed from preceding positions on $\mathbf{p}_i(t)$, i.e., if $\psi(\mathbf{p}_i(t_k), \mathbf{p}_f) \geq \psi_{min}$ for some time $t_k \leq t_f$. The best case is that for all $t \leq t_f$ the visibility level is higher than ψ_{min}, in which no modification to the trajectory is needed (Line 3-4). However, in cluttered environments $\mathbf{p}_f$ may not be observable until some time t_c, which means $\psi(\mathbf{p}_i(t), \mathbf{p}_f) < \psi_{min}$ when $t < t_c$ and $\psi(\mathbf{p}_i(t), \mathbf{p}_f) \geq \psi_{min}$ when $t \geq t_c$. Here, the view direction $\mathbf{v}_c = \frac{\mathbf{p}_c - \mathbf{p}_f}{\|\mathbf{p}_c - \mathbf{p}_f\|}$ where $\mathbf{p}_c = \mathbf{p}_i(t_c)$ is a critical view direction, in which $\psi(\mathbf{p}_i(t), \mathbf{p}_f)$ just reaches the desired level ψ_{min}. In this case **CriticalView**() reports $\mathbf{v}_c$ and the corresponding position $\mathbf{p}_c$ and time t_c.

B. Iterative Refinement

As is the worst case, an unknown obstacle may be revealed right behind $\mathbf{p}_f$ and block the trajectory. To ensure safety, we refine $\mathbf{p}_i(t)$ so that under the worst case the quadrotor will be able to avoid collision by taking some maneuvers whose single-axis acceleration does not exceed the limit a_{max}. We

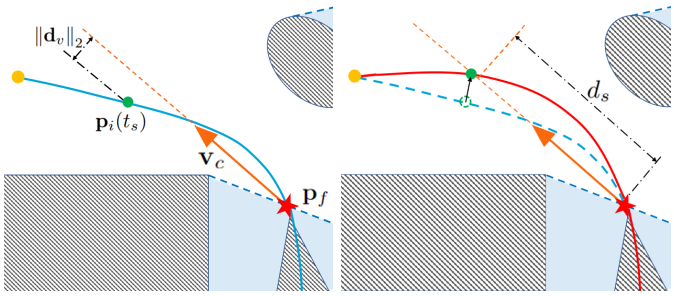


Fig. 11. The initial trajectory (blue) gets refined with the visibility status. Along the refined trajectory (red) $\mathbf{p}_f$ will become viewable earlier, wherein the reaction distance for collision avoidance is also increased.

Algorithm 3: Risk-aware Trajectory Refinement.

1 $\mathbf{p}_f, t_f \leftarrow$ **FrontierIntersection**$(\mathbf{p}_i(t))$
2 $visible, \mathbf{v}_c, \mathbf{p}_c, t_c \leftarrow$ **CriticalView**$(\mathbf{p}_i(t), \mathbf{p}_f)$
3 **if** $visible == True$ **then**
4 **return** $\mathbf{p}_i(t)$
5 $v_c \leftarrow \|\dot{\mathbf{p}}_i(t_c)\|_2$, $d_{cf} \leftarrow \|\mathbf{p}_c - \mathbf{p}_f\|_2$
6 **if** $v_c^2/2a_{max} \leq d_{cf} - R_q$ **then**
7 **return** $\mathbf{p}_i(t)$
8 $\hat{v}_s \leftarrow$ **AverageSpeed**$(\mathbf{p}_i(t), t_0, t_f)$
9 **repeat**
10 $\hat{d}_s \leftarrow \hat{v}_s^2/2a_{max} + R_q$
11 $t_s \leftarrow \arg\min_t f_{\mathbf{d}_v} + w_r f_{\mathbf{d}_{sf}}$
12 $\mathbf{p}_r(t) \leftarrow$ **RefineTrajectory**$(\mathbf{p}_i(t), \mathbf{p}_f, \mathbf{v}_c, t_s, \hat{d}_s)$
13 $v_s \leftarrow \|\dot{\mathbf{p}}_r(t_s)\|_2$, $d_{sf} \leftarrow \|\mathbf{p}_r(t_s) - \mathbf{p}_f\|_2$
14 $\hat{v}_s \leftarrow \alpha \cdot \hat{v}_s$
15 **until** $v_s^2/2a_{max} \leq d_{sf} - R_q$
16 **return** $\mathbf{p}_r(t)$

first check whether $\mathbf{p}_i(t)$ satisfies the worst-case safety criteria (Line 5-7). If it does not, the critical view and safe reaction distance constraints are enforced (Line 8-15).

1) Worst-case Safety Criteria: As showed in Sect.VI-A3, initially $\mathbf{p}_f$ will not become reliably viewable until t_c at $\mathbf{p}_c$. Suppose at $\mathbf{p}_c$ the speed is v_c, while the distance to $\mathbf{p}_f$ is d_{cf}. Similar to [27], we check if Equ.6 holds:

$$v_c^2/2a_{max} \leq d_{cf} - R_q \tag{6}$$

which means that if at $\mathbf{p}_c$ the quadrotor sees an obstacle, it can decelerate to a stop before colliding with the obstacle right behind $\mathbf{p}_f$. R_q compensates the quadrotor size and disturbance. If it is not true, extra constraints are added to meet this criteria.

2) View and Safety Constraints: If initially it would be too late for collision avoidance, $\mathbf{p}_f$ should be viewed at an earlier stage $t_s < t_c$, so that maneuvers to avoid collision can be taken in advance. Given the critical view direction $\mathbf{v}_c$, this constraint can be expressed as:

$$\|\mathbf{d}_v\|_2 \leq \delta_v \tag{7}$$

$$\mathbf{d}_v = (\mathbf{p}_i(t_s) - \mathbf{p}_f) - ((\mathbf{p}_i(t_s) - \mathbf{p}_f) \cdot \mathbf{v}_c)\,\mathbf{v}_c \tag{8}$$

The interpretation is that at t_s the quadrotor should have already reached the ray $\{\mathbf{q} \,|\, \mathbf{q} = \mathbf{p}_f + \lambda \mathbf{v}_c, \lambda \geq 0\}$ from which $\mathbf{p}_f$ is viewable (Fig.11). Besides the advanced observation, the distance from $\mathbf{p}_r(t_s)$ to $\mathbf{p}_f$ should be sufficient for an emergency stop:

$$d_{sf} = \|\mathbf{p}_i(t_s) - \mathbf{p}_f\|_2 \geq d_s \tag{9}$$

where d_s is the safe reaction distance that depends on the speed v_s at the refined trajectory $\mathbf{p}_r(t_s)$: $d_s = v_s^2/2a_{max} + R_q$. However, since the refined trajectory is not obtained yet, v_s is not available at this stage. To solve this chicken-and-egg problem, we introduce an iterative strategy. At the beginning we use the average speed of the segment of $\mathbf{p}_i(t)$ between $[t_0, t_f]$ as an estimate of v_s (Line 8), where t_0 is the start time of $\mathbf{p}_i(t)$. Then the trajectory is refined with the view and safety constraints (Line 10-12). After the refinement we check whether the new trajectory satisfies the safety criteria. If it does not, we increased the estimated speed $\hat{v}_s$ with a factor α slightly larger than 1 and redo the refinement (Line 13-15). This strategy is complete because it only terminates after the safety criteria is indeed met. It also terminates quickly, since the speed along the smooth trajectory varies slightly and it only takes one or a few steps to find a good estimate of v_s in practice.

RefineTrajectory() is essentially optimizing the trajectory with the newly introduced view and safety constraints. To incorporate them into our gradient-based optimization (Sect.IV-B), Equ.7-9 are soft-constrained by penalty functions:

$$f_{\mathbf{d}_v} = \mathcal{F}(\delta_v, \|\mathbf{d}_v\|_2) \tag{10}$$

$$f_{\mathbf{d}_{sf}} = \mathcal{F}(\|\mathbf{p}_i(t_s) - \mathbf{p}_f\|_2, d_s) \tag{11}$$

Here $\mathcal{F}()$ is the penalty function (Equ.3). The Jacobian of these terms are:

$$\frac{\partial f_{\mathbf{d}_v}}{\partial \mathbf{q}_k} = \begin{cases} \dfrac{2(\|\mathbf{d}_v\|_2 - \delta_v)\mathbf{d}_v^{\mathrm{T}}}{\|\mathbf{d}_v\|_2}(\mathbf{I} - \mathbf{v}_c\mathbf{v}_c^{\mathrm{T}})\,\dfrac{\partial \mathbf{p}_i(t_s)}{\partial \mathbf{q}_k}, & \|\mathbf{d}_v\|_2 > \delta_v \\ 0 & \text{, else} \end{cases} \tag{12}$$

$$\frac{\partial f_{\mathbf{d}_{sf}}}{\partial \mathbf{q}_k} = \begin{cases} 0 & , \|\mathbf{p}_i(t_s) - \mathbf{p}_f\|_2 \geq d_s \\ \dfrac{2(\|\mathbf{p}_i(t_s) - \mathbf{p}_f\|_2 - d_s)\mathbf{p}_i(t_s)^{\mathrm{T}}}{\|\mathbf{p}_i(t_s) - \mathbf{p}_f\|_2}\,\dfrac{\partial \mathbf{p}_i(t_s)}{\partial \mathbf{q}_k}, & \text{else} \end{cases} \tag{13}$$

Here $\mathbf{q}_k$ B-spline control points associated with $\mathbf{p}_i(t_s)$. The refinement minimizes the cost function:

$$f_r = \underbrace{\lambda_s f_s + \lambda_c f_c + \lambda_d (f_v + f_a)}_{f_{p2}} + \lambda_r (f_{\mathbf{d}_v} + w_r f_{\mathbf{d}_{sf}}) \tag{14}$$

where f_{p2} is exactly the same as Equ.2. Note that applying the constraints to different t_s generates different results. However, including t_s into the optimization to find its best value would make the problem too difficult to be solved quickly. For simplicity, we first determine t_s as the time that minimizes $f_{\mathbf{d}_v} + w_r f_{\mathbf{d}_{sf}}$ for the initial $\mathbf{p}_i(t)$ and use it for the refinement (Line 11-12). Quantitatively, the selected t_s leads to minimal violation of the constraints, therefore enforcing the constraints at t_s also requires less modification to $\mathbf{p}_i(t)$, which is a reasonably good choice.

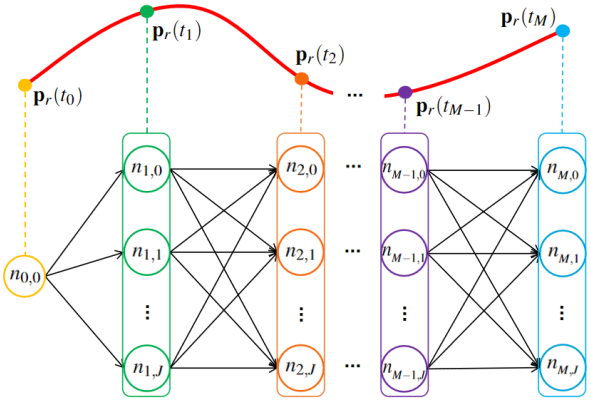


Fig. 12. The directed graph modeled for searching a sequence of yaw angles along the refined trajectory. Each layer of nodes (with the same color) is associated with a position on the trajectory.

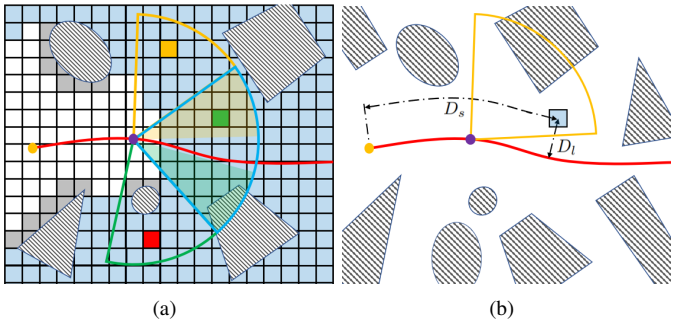


Fig. 13. (a) The evaluation of potential information gain. Blue transparent voxels represent unknown areas. Occupied and free space is represented by gray and white colors. The yellow voxel will contribute to the information gain, while the red voxel will not since it is blocked by occupied voxels. The green voxel is inside the overlapping region of multiple FOVs, whose visibility is stored after the first raycasting and queried directly in subsequent check. (b) Weighting a unknown and visible voxel by its lateral and longitudinal distance to the trajectory.

VII. Yaw Angle Planning

Quadrotors typically have limited sensor FOVs. To improve flight safety, we plan the trajectory of yaw angle to actively observe the environments. Inspired by the recent two-step quadrotor motion planning paradigm [4]–[8, 15], we also decompose the yaw angle planning into a graph search problem and a trajectory optimization.

A. Graph Search

1) Problem Modeling: We model a graph search problem to seek for an sequence of yaw angles $\Xi := \{\xi_0, \xi_1, \cdots, \xi_M\}$ along the refined trajectory that trades off the smoothness and information gain (IG, detailed in Sect.VII-A2) of the unknown space. Given the quadrotor trajectory $\mathbf{p}_r(t)$ (Sect.VI), a set of positions $\mathbf{p}_{r,i}, i \in [0, 1, \cdots, M]$ uniformly distributed along the trajectory at $\{t_0, t_1, \cdots, t_M\}$ are selected. At $\mathbf{p}_i$ expect $i = 0$, where the yaw angle is already determined by the current quadrotor's state, several graph nodes $n_{i,j}, j \in [0, 1, \cdots, J]$ are created, each of which associates a different angle $\xi_{i,j}$ and the IG $g_{i,j}$ at the state $(\mathbf{p}_{r,i}, \xi_{i,j})$. For each pair of nodes n_{i,j_1}, n_{i+1,j_2} associated with adjacent positions, a graph edge from n_{i,j_1} to n_{i+1,j_2} is created. This process construct a directed graph as shown in Fig.12. To find a sequence of yaw that maximizes IG and smoothness, we search a path in the graph that minimizes the cost $c(\Xi)$, which can be solved efficiently by the Dijkstra algorithm:

$$c(\Xi) = \sum_{i=1}^{M} \left\{ -g_{i,j_i} + \mu \left(\xi_{i,j_i} - \xi_{i-1,j_{i-1}} \right)^2 \right\} \qquad (15)$$

where μ is used to adjust the weighting of smoothness.

2) Information Gain: We employ a similar method to [44] which assesses potential IG as the number of unmapped voxels that comply with the camera model and are visible (not blocked by occupied voxels). However, the original method does raycasting for every voxels inside the camera FOV to validate their visibility, which is too expensive to function online. Therefore, we adapt it to better suit the real-time planning in several ways: (a) As is in [38], voxels inside the FOV are subsampled to approximate the actual gain, which leads to only slight error but great run time reduction. (b) The gains of different $\xi_{i,j}$ are evaluated in parallel. (c) We borrow the techniques from [45] to avoid repeated raycasting. As depicted in Fig.13(a), we notice that at one position $\mathbf{p}_{r,i}$ where different $\xi_{i,j}$ are assessed, many voxels are in overlapping areas and are checked for visibility more than once. To avoid unnecessary repetition, we store the visibility of each voxel when it is checked for the first time, so that in subsequent check the visibility and be queried directly. In these ways, the overall IG evaluation time is reduced by over two orders of magnitude.

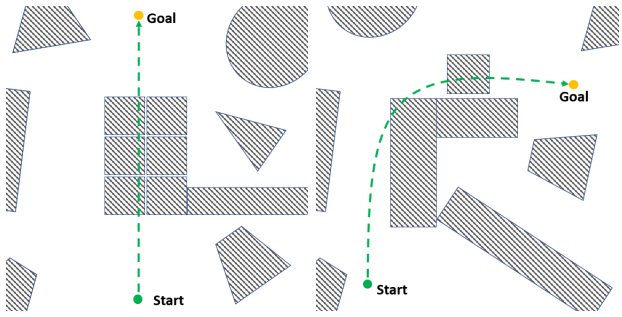
In the context of exploration [44], every voxel contributes equally to the IG of one quadrotor configuration, ensuring that all space can be covered by the sensors uniformly. However, in a point-to-point navigation we do not aim at full coverage but prefer focusing on space relevant to the flight. In particular, unknown voxels closer to the trajectory and the current position have higher influence to the flight. Therefore, we use Equ.16 to bias the IG contribution Ig of a visible voxel centered at $\mathbf{m}$:

$$Ig(\mathbf{m}) = \exp\left\{ -w_l \, D_l(\mathbf{m}, \mathbf{p}_r(t)) - w_s \, D_s(\mathbf{m}, \mathbf{p}_r(t)) \right\} \qquad (16)$$

where $D_l(\mathbf{m}, \mathbf{p}_r(t))$ and $D_s(\mathbf{m}, \mathbf{p}_r(t))$ represent the lateral and longitudinal distance to trajectory $\mathbf{p}_r(t)$ (Fig.13(b)).

B. Yaw Angle Optimization

Given the optimal path Ξ searched through the graph, we compute the trajectory of yaw angle $\phi(t)$ that is smooth, dynamically feasible and passes through the sequential angles ξ_j. We parameterize $\phi(t)$ as a uniform B-spline with control points $\Phi := \{\phi_{c,0}, \phi_{c,1}, \cdots, \phi_{c,N_c}\}$ and knot span δt_ϕ. In



(a) (b)

Fig. 14. An illustration of the two real-world scenarios for testing the perception-aware replanning. (a) A large obstacles is placed along the straight-line global reference trajectory. (b) An obstacles is placed right behind the corner.

this way, the convex hull property can be employed to ensure dynamic feasibility [15]. The problem is formulated as:

$$\arg\min_{\Phi} \int_{t_0}^{t_M} (\dddot{\phi}(t))^2 dt + \gamma_1 \sum_{i=0}^{M} (\phi(t_i) - \xi_i)^2 \tag{17}$$
$$+ \gamma_2 \left\{ \sum_{j=0}^{N_c-1} \mathcal{F}(\dot{\phi}_{max}, |\dot{\phi}_{c,j}|) + \sum_{j=0}^{N_c-2} \mathcal{F}(\ddot{\phi}_{max}, |\ddot{\phi}_{c,j}|) \right\}$$

Here the first term represents smoothness and the second term is a soft waypoint constraint enforcing $\phi(t)$ to pass through Ξ. The last two terms are the soft constraints for dynamic feasibility, wherein $\dot{\phi}_{c,j}$ and $\ddot{\phi}_{c,j}$ are the B-spline control points of angular velocity and acceleration:

$$\dot{\phi}_{c,j} = \frac{\phi_{c,j+1} - \phi_{c,j}}{\delta t_\phi}, \quad \ddot{\phi}_{c,j} = \frac{\phi_{c,j+2} - 2\phi_{c,j+1} + \phi_{c,j}}{\delta t_\phi^2} \tag{18}$$

Thanks to the convex hull property of B-spline, the entire trajectory is guaranteed to be feasible given that the control points do not exceed the dynamic limits $\dot{\phi}_{max}, \ddot{\phi}_{max}$.

VIII. Results

A. Implementation Details

We present tests in both real world and simulation. In real-world experiments, a customized quadrotor platform equipped with an Intel RealSense Depth Camera D435 is used. All the state estimation, mapping, planning and control modules run on an Intel Core i7-8550U CPU. For simulation, we use a simulating tool containing the quadrotor dynamics model, random map generator and depth image renderer. The dynamics model relies on a numeric ODE solver *odeint*[4]. The depth images are rendered in GPU by projecting point cloud of the surrounding obstacles onto the image plane. Random noises are added to them to better mimic the real measurements. All simulations run on an Intel Core i7-8700K CPU and GeForce GTX 1080 Ti GPU. The trajectory optimization is solved by a general non-linear optimization solver NLopt[5].

[4]www.boost.org/doc/libs/1_73_0/libs/numeric/odeint/doc/html/index.html
[5]https://nlopt.readthedocs.io/en/latest/

TABLE I
Four planners tested in real-world experiments.

Planner	Strategy	Success Number	
		Scene 1	Scene 2
A	Optimistic & Velocity-tracking	0	0
B	Optimistic & Active exploration	0	0
C	Risk-aware & Velocity-tracking	2	0
D	Risk-aware & Active exploration	**3**	**3**

1) Global Planning: We use the approach [3] to compute global reference trajectories. Note that we focus on evaluating the local replanning system, therefore, naive global trajectories are given, such as straight-line trajectory connecting the start and goal positions.

2) Volumetric Mapping: In all tests, the quadrotor starts with no prior knowledge of the environments. A volumetric mapping framework [46] fuses the depth images from the stereo camera into a occupancy grid map. An ESDF is derived from the occupancy grid map using an efficient distance transform algorithm [47] to support the gradient-based optimization (Sect.IV) and visibility evaluation (Sect.VI-A2). Trilinear interpolation is also used to reduce the distance error induced by the discrete grid map.

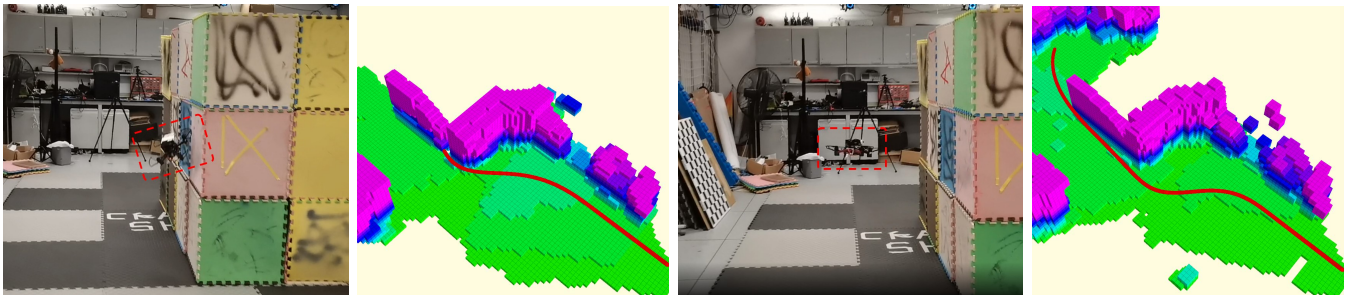
3) State Estimation and Control: We localize the drone by a robust visual-inertial state estimator [48] in real-world tests. In simulation, ground truth odometry is generated by the quadrotor dynamics model. We use a geometric controller [49] to track both the position and yaw trajectory.

The following evaluation is divided into two parts. The first part evaluates the perception-aware planning strategy, the second part tests the whole replanning framework.

B. Perception-aware Planning Strategy

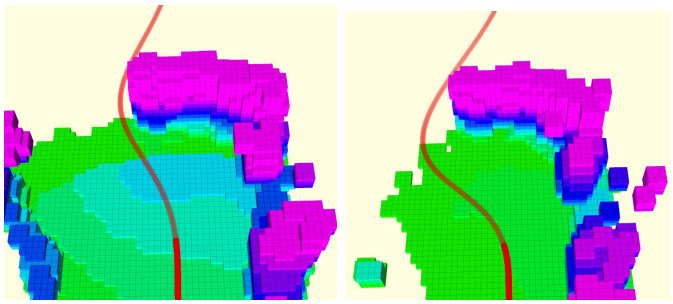
1) Real-world Tests: We conduct comparative experiments to show the importance of introducing active perception. Specifically we compare the proposed strategy: the risk-aware refinement (Sect.VI) and active exploration yaw (Sect.VII) with the commonly used ones: the optimistic assumption and the velocity-tracking yaw. Optimistic assumption treats all unknown space as collision-free, which is frequently adopted such as in [7, 14, 15, 26]. The velocity-tracking yaw relates the desired yaw angle to the velocity: $\phi(t) = \arctan(\frac{v_y(t)}{v_x(t)})$, to increase the chance of seeing obstacles. Four local planners listed in Tab.I are tested in two scenes. Each planner is tested 3 times in both scenes and we record the number of successful flights. The maximum velocity and acceleration are set as $3m/s$ and $2.5m/s^2$. Whenever collision along the trajectory is detected and the collision point is closer than $0.5m$, emergency stop is conducted immediately for safety.

In the first scene, a straight-line global reference trajectory is given (Fig.14(a)). A large obstacle consisting of several boxes and boards are placed on the way. When the quadrotor approaches the obstacle, boxes in the front row will be revealed first, while others behind them are occluded and invisible at the beginning. Planners with optimistic assumption (A & B) are unaware of the potential danger behind. They simply replan trajectories to avoid the viewed boxes, along which there is low visibility to the boxes in the back, as showed in Fig.16(a). As



(a) (b) (c) (d)

Fig. 15. Comparison of four local planners in scene 1. (a) using planners A and B, the boxes in the back row are not discovered until the quadrotor gets very close. Consequently the quadrotor has to pause in emergency. (b) The 3D maps (colorful voxels) and executed trajectories (red curves) at the moment when **boxes in the back are just discovered**. (c),(d): the quadrotor avoid all boxes successfully with planners D. More details are showed in the video.



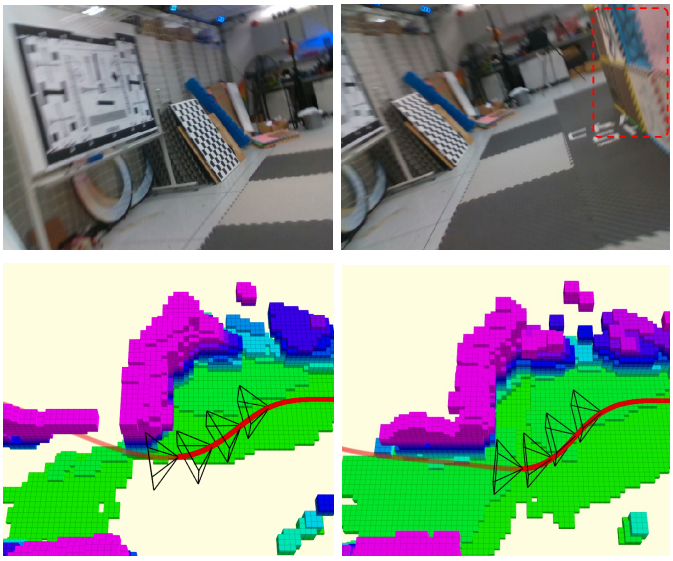
(a) (b)

Fig. 16. A comparison of the trajectories in scene 1 replanned with (a) optimistic assumption and (b) risk-aware refinement. Trajectories already executed and not executed yet are showed in opaque red and transparent red respectively. (a) Along the trajectory visibility toward the unknown region behind the observed obstacle is poor. (b) The trajectory deviates more laterally, therefore along it there is greater visibility toward the unknown area in the back.

a result, the quadrotor gets 'surprised' by the occluded boxes afterwards and pauses in emergency, as showed in Fig.15(a) and 15(b). In contrast, planners with risk awareness (C & D) generate trajectories that deviate a bit more laterally, along which visibility toward the unknown area in the back is higher (Fig.16(b)). Therefore in both cases the quadrotor reach the goal more times. However, with the velocity-tracking yaw (planner C), the quadrotor does not face toward the unknown area in the back quickly, which postpones the discovery of occluded boxes and causes 1 failure. In comparison, with the active exploration yaw (planner D) the quadrotor quickly turns toward the unknown area and observes the previously occluded boxes, enabling itself to take action earlier. This comparison is displayed in Fig.17.

In the second scene, an obstacle is placed right behind the corner, which is invisible to the quadrotor until it turns right. The reference trajectory is set to pass through the obstacles deliberately (Fig.14(b)). In this scene, the quadrotor can only reach the goal safely with planner D. For other three planners, the quadrotor collides with the obstacle behind the corner, due to either the poor visibility of the replanned trajectories (planner A & B), or the delay of perception caused by the velocity-tracking yaw (planner C). Note that even emergency stop is conducted, the quadrotor fail to avoid collision in time (Fig.18). The comparisons of the replanned trajectories and yaw angle are displayed in Fig.19 and Fig.20 respectively.

The experiments demonstrate two critical factors to survive

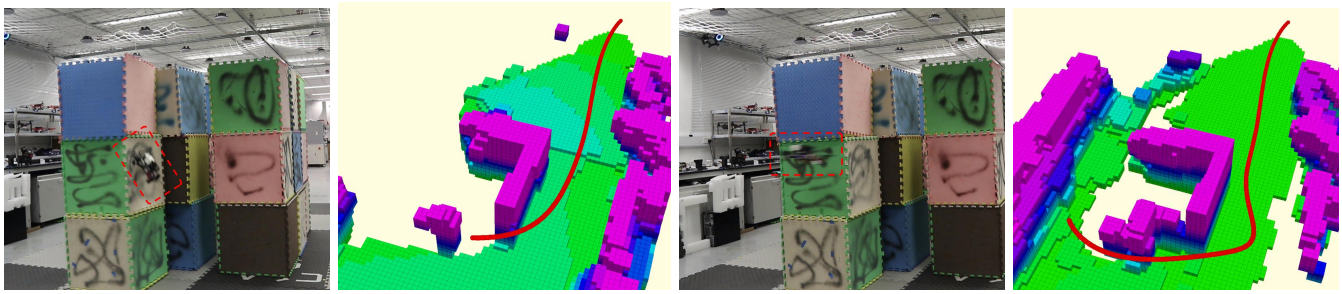


(a) (b)

Fig. 17. A comparison in scene 1 of (a) velocity-tracking yaw and (b) active exploration yaw. Top: The first person view (FPV) images. Bottom: the 3D maps and trajectories associated with the FPV images. The camera poses within 0.9 seconds (interval $= 0.3s$) are also visualized. (a) The quadrotor face to the left side at the beginning, so unknown boxes on the right side are not mapped completely. (b) The quadrotor discovers the previously occluded boxes earlier.

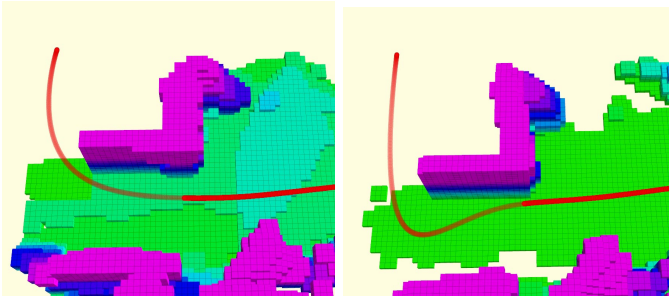
in high-speed flights: (a) having good visibility toward the unknown regions that will influence the flight and (b) looking toward the relevant direction to eliminate those unknown regions actively. The proposed method takes into account these factors and guarantees safety for fast flight. More details about the experiments are presented in the attached video.

2) Benchmark Comparisons: We compare the proposed strategy with the safe local exploration (SLE) presented in [38] in simulation. This strategy originated from the "next-best-view" planner [44] in the exploration literature, but is adapted for online functioning in goal reaching tasks. It repeatedly selects intermediate goals that are closer to the final goal and have higher information gain, after which a local planner replans new trajectories toward the goals. It also adopts the velocity-tracking yaw, as is detailed in Sect.VIII-B1. To compare the strategies fairly, we integrated both of them with our robust optimistic replanning (Sect.IV, V). They are tested in 10 random maps with 5 different obstacle densities, 5



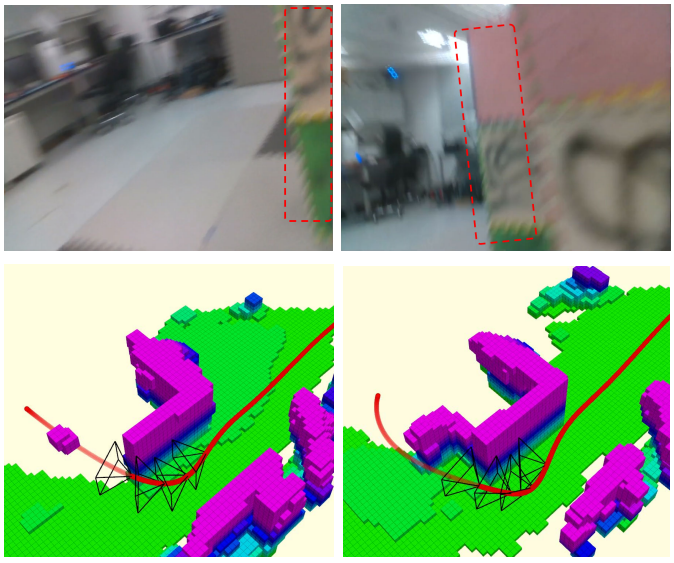
(a) (b) (c) (d)

Fig. 18. Tests in scene 2. (a),(b) The quadrotor fails to react to the obstacle after turning right and crashes. (c),(d) The quadrotor avoid the obstacle behind the corner successfully.



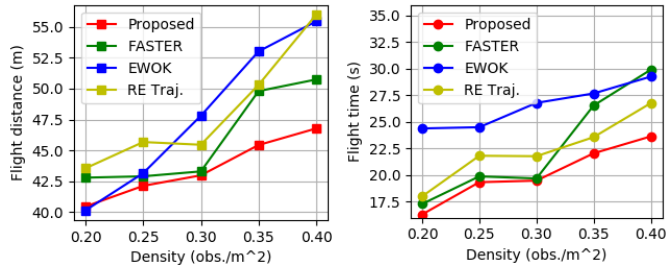
(a) (b)

Fig. 19. A comparison of the trajectories in scene 2 with (a) optimistic assumption and (b) risk-aware refinement. (a) The trajectory is close to the corner. (b) The trajectory is slightly longer around the corner, where the quadrotor can observe the unknown space in the back and react to possible dangers.



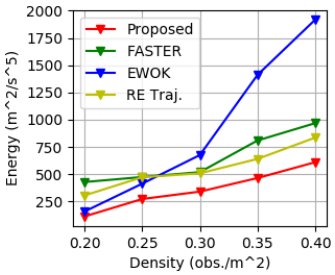
(a) (b)

Fig. 20. A comparison in scene 2 of (a) velocity-tracking yaw and (b) active exploration yaw. (a) The quadrotor turns its head to the right later. The hidden obstacles is not observed until getting close. (b) The quadrotor looks to the right side earlier and discovers the obstacle behind the corner.

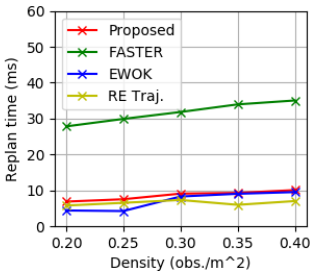


(a) Avg. flight distance (m).

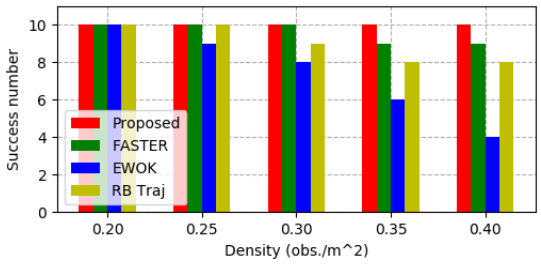
(b) Avg. flight time (s).



(c) Avg. consumed energy (m^2/s^5).



(d) Avg. replan time (ms).



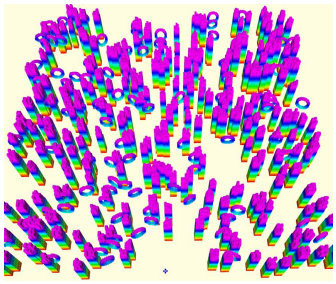
(e) Number of successful flights.

Fig. 21. Results of the comparisons between the proposed method and FASTER, EWOK and RE Traj..

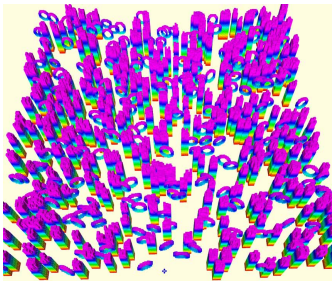
trials are conducted for each map. We compare the number of successful flight, flight time and flight distance. Samples of the maps are displayed in Fig.22(a), 22(b).

As is shown in Tab.II, the proposed strategy achieves higher number of successful flights when the scene gets more cluttered. Our strategy enforces visibility to dangerous unknown areas, and control the yaw angle to observe those areas actively. Therefore it can guarantee safety even the environment becomes very complex. For SLE, the velocity-tracking yaw is the major cause of failure, as it may not face toward dangerous unknown regions in time, as has been shown in Sect.VIII-B1.

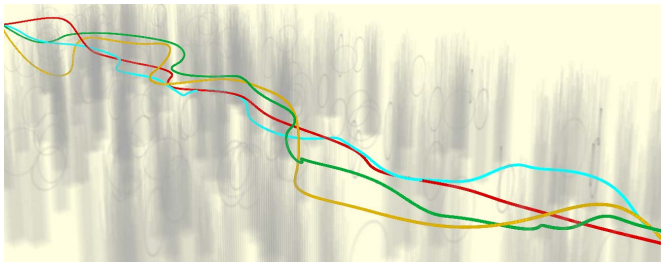
Besides, our strategy is also more beneficial to achieve lower flight distance and time than SLE. SLE only selects intermediate goals and plans within the known unoccupied space, which is conservative. Besides, since the selection



(a) Sample map 1: 0.25 obs./m^2



(b) Sample map 2: 0.4 obs./m^2



(c) Trajectories generated by the proposed method (red), FASTER [10] (green), EWOK [14] (cyan) and RE Traj. [15] (yellow). Obstacles are set as gray transparent for clarity.

Fig. 22. Samples of the maps and generated trajectories.

of intermediate goals takes information gain into account, the quadrotor tends to take some detours to gather more information, which leads to longer flight distance and time. In contrast, our strategy plans in both the known and unknown space, allowing more aggressive behaviors under the premise of safety. Moreover, instead of treating all unknown areas equally, it only focus on observing areas that are more relevant to the flight, which eliminates many unnecessary detours and improve the overall flight efficiency.

C. Replanning Framework for Fast Flight

1) Benchmark Comparisons: We compare our replanning framework with several state-of-the-art methods, FASTER [10], EWOK [14] and RE Traj. [15]. FASTER belongs to the hard-constrained category (Sect.II-A), and features maintaining a feasible and safe back-up trajectory in the free-known space at each replanning step to improve safety. It also adopts a mixed integer quadratic program (MIQP) formulation to obtain a more reasonable time allocation of the trajectories. Both [14, 15] belong to the gradient-based methods. They utilize a uniform B-spline trajectory representation to replan efficiently. [15] further exploits the convex hull property of B-spline and introduces a kinodynamic path searching to find more promising initial trajectories. We also test the four methods in 10 random maps with 5 obstacle densities. Note that all benchmarked methods are open-source and we use their default parameter settings. The number of successful flights, average flight distance, flight time, energy (integral of squared jerk), computation time of each replanning, and total replan number in each flight are recorded. Samples of the maps and the trajectories generated by the four methods are shown in Fig.22.

TABLE II
COMPARISONS OF THE PERCEPTION-AWARE PLANNING STRATEGIES.

Density (obs./m^2)	**Method**	**Number of Success**	**Flight Distance (m)**	**Flight Time (s)**
0.2	SLE [38]	10	43.475	20.012
	Proposed	10	**41.822**	**16.608**
0.25	SLE [38]	10	44.682	20.593
	Proposed	10	**42.133**	**19.338**
0.3	SLE [38]	9	44.874	22.316
	Proposed	**10**	**42.982**	**19.483**
0.35	SLE [38]	9	46.093	23.577
	Proposed	**10**	**45.436**	**22.050**
0.4	SLE [38]	8	47.817	27.903
	Proposed	**10**	**46.776**	**23.639**

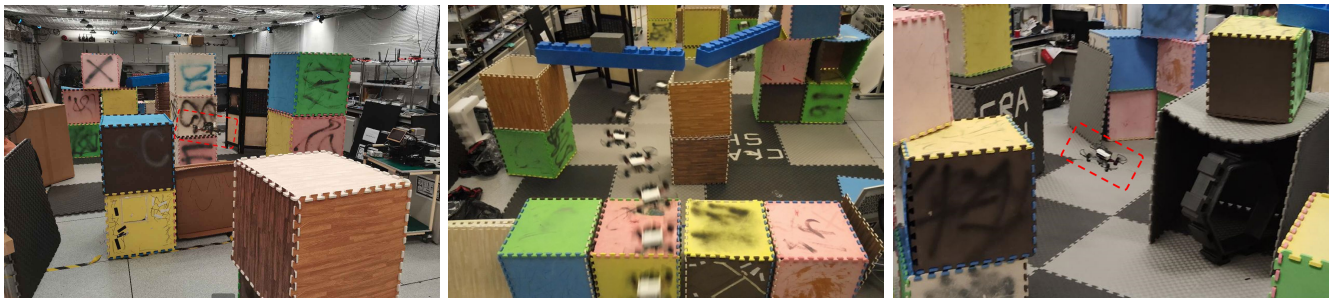
As displayed in Fig.21, our method outperform others in the aspects of flight distance, flight time and energy consumption, with competitive computation efficiency. FASTER rarely fails in the tests, thanks to the back-up trajectories. However, due to the computationally demanding MIQP formulation, its overhead is higher. The other two benchmarked methods are more efficient. However, EWOK suffers from the local minima issue, so it usually fails or outputs low-quality solutions in dense environments. The kinodynamic path searching and B-spline optimization adopted by RE Traj. relieve the local minima significantly. Nonetheless, due to the lack of perception consideration, the succuss number is mediocre in dense environments. Compared to them, the proposed method search the solution space effectively with the guidance of topologically distinctive paths, and generates high-quality trajectories consistently. Safety is also reenforced by introducing perception awareness.

2) Indoor Flight Test: We conduct aggressive flight experiments in three indoor scenes (Fig.2, 23) to validate our planning system. Various types of obstacles are placed randomly and densely to make up the challenging flight environments. Distance of neighboring obstacles are only around 1 meter, making the space for safe navigation very limited. Besides, the high obstacle density makes visibility to the environment very restricted, since many obstacles are occluded by others, which poses greater challenges to the replanning algorithm.

In each experiment, the final goal is set to $14m$ away from the quadrotor. Straight-line global reference trajectories are given and local replanning is conducted within a horizon of 7 m. Samples of the online generated map and executed trajectories are presented in Fig.24. The velocity profile of one flight are showed in Fig.25(a), in which the maximum speed is 2.90 m/s and average speed is 1.77 m/s. The flight distance and time are 14.12 m and 8.0 s respectively. We refer the readers to the attached video for more tests.

3) Outdoor Flight Test: Finally, we conduct fast flight tests in three different outdoor scenes, as displayed in in Fig.26, to validate our planning method in natural environments. The outdoor environments are typically unstructured and irregular, where the quadrotor should perform agile 3D maneuvers to avoid obstacles such as rocks and branches and leaves of trees. Note that despite the outdoor environments, we do not use external devices for localization.

Results of the online generated map and executed trajecto-

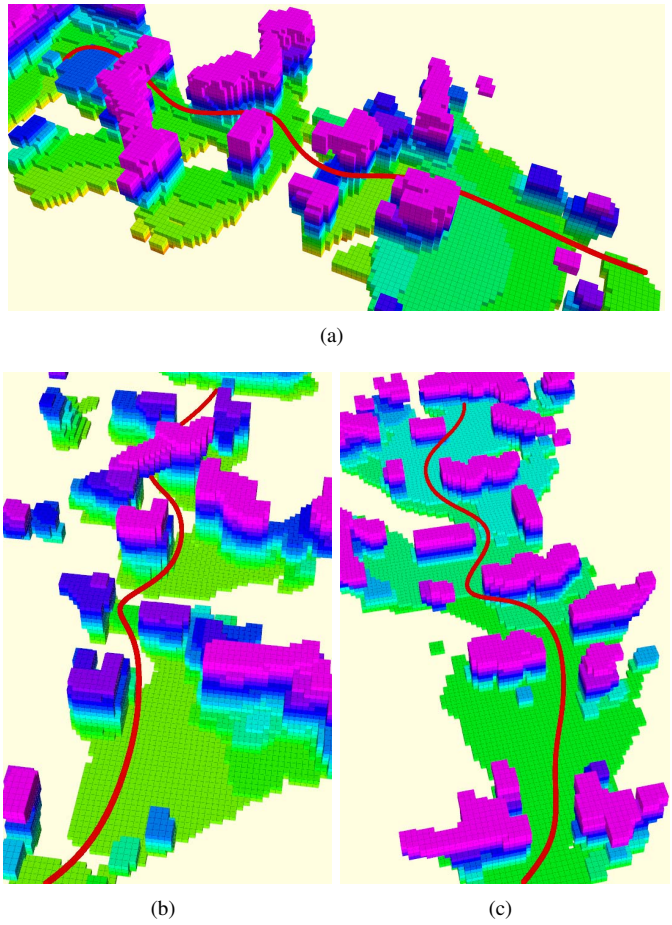


(a) The quadrotor passes a horizontal cardboard, after which it will avoid the vertical pillar.

(b) Composite image of the flight experiment.

(c) The quadrotor flies in the narrow passages and avoids boxes.

Fig. 23. Three indoor scenes for the fast flight experiments. Several tests are conducted for each scenes, more details are available in the attached video.

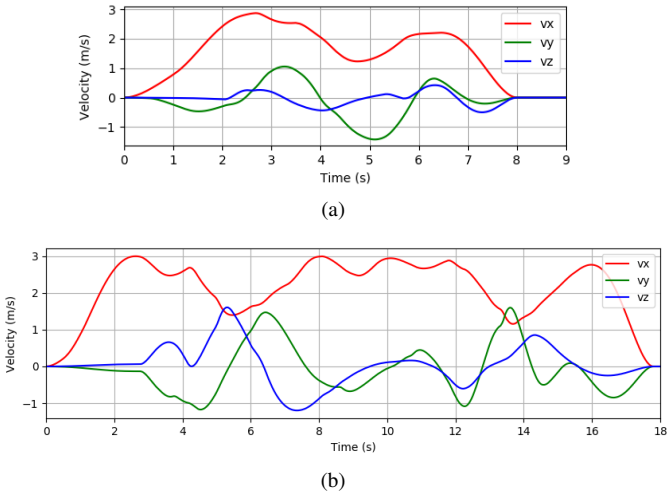


(a)

(b) (c)

Fig. 24. Samples of the online generated maps and trajectories executed by the quadrotor in the different indoor scenes.

ries are presented in Fig.27. In the first scene, the quadrotor flies through the forest to the goal 39 m away from the initial position. The velocity profile is showed in Fig.25(b). The maximum speed is 3.19 m/s and average speed is 2.29 m/s. The flight takes 40.78 m and 17.83 s. In the second scene, the quadrotor flies up a slope to the first goal, after which it flies to the second goal. The first goal is 30 m away and the change in height is 7 m. The second goal is 17 m far. The whole flight takes 48.71 m and 23.19 s. The third scene is a larger forest, where the goal is set to 45 m away. The flight distance is 46.85 m, which takes 21.91 s to finish. The maximum and



(a)

(b)

Fig. 25. Velocity profiles of sample flights in (a) indoor experiment (b) forest experiment.

average speed are 3.41 m/s and 2.14 m/s. More details of the flights are showed in the attached video.

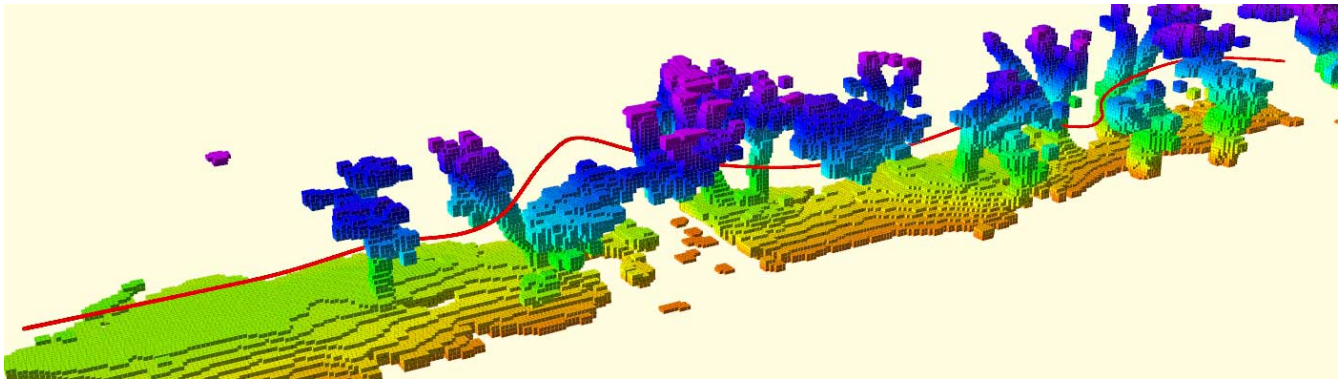
IX. Conclusions

In this paper, we propose a robust and perception-aware replanning method for high-speed quadrotor autonomous navigation. The path-guided optimization and topological path searching are devised to escape from local minima and explore the solution space more thoroughly, through which higher robustness and optimality guarantee are obtained. The robust planner is further enhanced by the perception-aware strategy, which takes special caution about regions that may be dangerous to the quadrotor. The yaw angle of the quadrotor is also planned to actively explore the environments, especially areas that are relevant to the future flight. The planning system is evaluated comprehensively through benchmark comparisons. We integrate the planning method with global planning, state estimation, mapping, and control into a quadrotor platform and conduct extensive challenging indoor and outdoor flight tests. Results show that the proposed method is robust and capable of supporting fast and safe flights. We release the implementation of our system to the community.

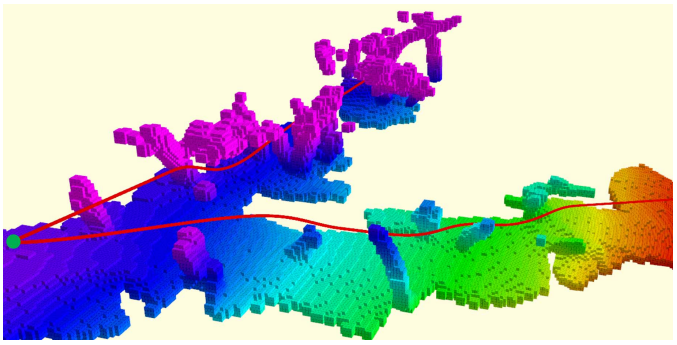


(a) Flying through forest 1. (b) Flying up the slope in forest 1. (c) Flying through forest 2.

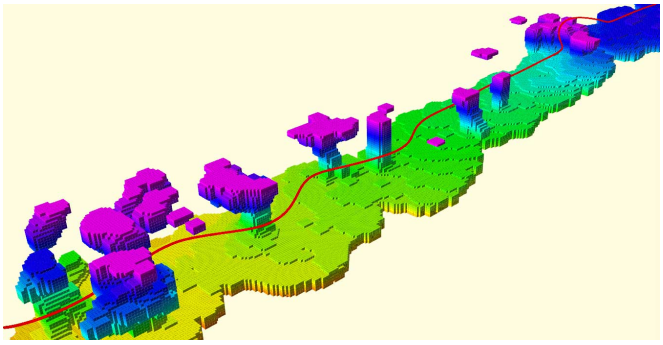
Fig. 26. Autonomous fast flight experiments in two dense forests.



(a) Flight in forest 1.



(b) The quadrotor flies up the slope to the first goal (green circle), after which it flies toward the second goal.



(c) Flight in forest 2.

Fig. 27. Online generated maps of the forests and trajectories executed by the quadrotor. The map is colored by height.

REFERENCES

[1] B. Zhou, F. Gao, J. Pan, and S. Shen, "Robust real-time uav replanning using guided gradient-based optimization and topological paths," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2020, to appear. [Online]. Available: https://arxiv.org/abs/1912.12644

[2] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, Shanghai, China, May 2011, pp. 2520–2525.

[3] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Proc. of the Intl. Sym. of Robot. Research (ISRR)*, Dec. 2013, pp. 649–666.

[4] J. Chen, K. Su, and S. Shen, "Real-time safe trajectory generation for quadrotor flight in cluttered environments," in *Proc. of the IEEE Intl. Conf. on Robot. and Biom.*, Zhuhai, China, Aug. 2015.

[5] F. Gao, W. Wu, Y. Lin, and S. Shen, "Online safe trajectory generation for quadrotors using fast marching method and bernstein basis polynomial," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, Brisbane, Australia, May 2018.

[6] F. Gao and S. Shen, "Online quadrotor trajectory generation and autonomous navigation on point clouds," in *Proc. of the IEEE Intl. Sym. on Safety, Security, and Rescue Robotics (SSRR)*, lausanne, switzerland, 2016, pp. 139–146.

[7] F. Gao, W. Wu, W. Gao, and S. Shen, "Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments," *Journal of Field Robotics*, 2018. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.21842

[8] W. Ding, W. Gao, K. Wang, and S. Shen, "An efficient b-spline-based kinodynamic replanning framework for quadrotors," *IEEE Transactions on Robotics*, vol. 35, no. 6, pp. 1287–1306, 2019.

[9] ——, "Trajectory replanning for quadrotors using kinodynamic search and elastic optimization," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2018, pp. 7595–7602.

[10] J. Tordesillas, B. T. Lopez, and J. P. How, "FASTER: Fast and safe trajectory planner for flights in unknown environments," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2019.

[11] H. Oleynikova, M. Burri, Z. Taylor, J. Nieto, R. Siegwart, and E. Galceran, "Continuous-time trajectory optimization for online uav replanning," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*, Daejeon, Korea, Oct. 2016, pp. 5332–5339.

[12] F. Gao, Y. Lin, and S. Shen, "Gradient-based online safe trajectory generation for quadrotor flight in complex environments," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*, Sept 2017, pp. 3681–3688.

[13] Y. Lin, F. Gao, T. Qin, W. Gao, T. Liu, W. Wu, and S. Shen, "Autonomous aerial navigation using monocular visual-inertial fusion," *Journal of Field Robotics*, vol. 35, no. 1, pp. 23–51, 2018.

[14] V. Usenko, L. von Stumberg, A. Pangercic, and D. Cremers, "Real-time trajectory replanning for mavs using uniform b-splines and a 3d circular buffer," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2017, pp. 215–222.

[15] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3529–3536, 2019.

[16] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "Chomp: Gradient optimization techniques for efficient motion planning," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, May 2009, pp. 489–494.

[17] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "Stomp: Stochastic trajectory optimization for motion planning," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 4569–4574.

[18] E. Schmitzberger, J.-L. Bouchet, M. Dufaut, D. Wolf, and R. Husson, "Capture of homotopy classes with probabilistic road map," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*, vol. 3. IEEE, 2002, pp. 2317–2322.

[19] C. Rösmann, F. Hoffmann, and T. Bertram, "Planning of multiple robot trajectories in distinctive topologies," in *2015 European Conference on Mobile Robots (ECMR)*. IEEE, 2015, pp. 1–6.

[20] ——, "Integrated online trajectory planning and optimization in distinctive topologies," *Robotics and Autonomous Systems*, vol. 88, pp. 142–153, 2017.

[21] S. Bhattacharya, "Search-based path planning with homotopy class constraints," in *Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010.

[22] S. Bhattacharya, M. Likhachev, and V. Kumar, "Topological constraints in search-based robot path planning," *Autonomous Robots*, vol. 33, no. 3, pp. 273–290, 2012.

[23] L. Jaillet and T. Siméon, "Path deformation roadmaps: Compact graphs with useful cycles for motion planning," *The International Journal of Robotics Research*, vol. 27, no. 11-12, pp. 1175–1188, 2008.

[24] H. Oleynikova, Z. Taylor, R. Siegwart, and J. Nieto, "Sparse 3d topological graphs for micro-aerial vehicle planning," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2018, pp. 1–9.

[25] F. Blochliger, M. Fehr, M. Dymczyk, T. Schneider, and R. Siegwart, "Topomap: Topological mapping and navigation based on visual slam maps," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1–9.

[26] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhattacharya, C. J. Taylor, and V. Kumar, "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments," *IEEE Robotics and Automation Letters (RA-L)*, pp. 1688–1695, 2017.

[27] S. Liu, M. Watterson, S. Tang, and V. Kumar, "High speed navigation for quadrotors with limited onboard sensing," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2016, pp. 1484–1491.

[28] B. T. Lopez and J. P. How, "Aggressive 3-d collision avoidance for high-speed navigation." in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, 2017, pp. 5759–5765.

[29] M. Nieuwenhuisen and S. Behnke, "Search-based 3d planning and trajectory optimization for safe micro aerial vehicle flight under sensor visibility constraints," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2019, pp. 9123–9129.

[30] B. T. Lopez and J. P. How, "Aggressive collision avoidance with limited field-of-view sensing," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2017, pp. 1358–1365.

[31] G. Costante, C. Forster, J. Delmerico, P. Valigi, and D. Scaramuzza, "Perception-aware path planning," *IEEE Trans. Robot. (TRO)*, pp. Epub–ahead, 2016.

[32] Z. Zhang and D. Scaramuzza, "Perception-aware receding horizon navigation for mavs," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2018, pp. 2534–2541.

[33] V. Murali, I. Spasojevic, W. Guerra, and S. Karaman, "Perception-aware trajectory generation for aggressive quadrotor flight using differential flatness," in *2019 American Control Conference (ACC)*. IEEE, 2019, pp. 3936–3943.

[34] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "Pampc: Perception-aware model predictive control for quadrotors," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2018, pp. 1–8.

[35] B. F. Jeon and H. J. Kim, "Online trajectory generation of a mav for chasing a moving target in 3d dense environments," *arXiv preprint arXiv:1904.03421*, 2019.

[36] C. Richter and N. Roy, "Learning to plan for visibility in navigation of unknown environments," in *International Symposium on Experimental Robotics*. Springer, 2016, pp. 387–398.

[37] E. Heiden, K. Hausman, G. S. Sukhatme, and A.-a. Agha-mohammadi, "Planning high-speed safe trajectories in confidence-rich maps," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2017, pp. 2880–2886.

[38] H. Oleynikova, Z. Taylor, R. Siegwart, and J. Nieto, "Safe local exploration for replanning in cluttered unknown environments for microaerial vehicles," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1474–1481, 2018.

[39] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *The International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, 2014.

[40] S. Liu, N. Atanasov, K. Mohta, and V. Kumar, "Search-based motion planning for quadrotors using linear quadratic minimum time control," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*, Sept 2017, pp. 2872–2879.

[41] D. J. Webb and J. van den Berg, "Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, Germany, May 2013, pp. 5054–5061.

[42] T. Siméon, J.-P. Laumond, and C. Nissoux, "Visibility-based probabilistic roadmaps for motion planning," *Advanced Robotics*, vol. 14, no. 6, pp. 477–493, 2000.

[43] Y.-H. Tsai, L.-T. Cheng, S. Osher, P. Burchard, and G. Sapiro, "Visibility and its dynamics in a pde based implicit framework," *Journal of Computational Physics*, vol. 199, no. 1, pp. 260–290, 2004.

[44] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon path planning for 3d exploration and surface inspection," *Auton. Robots*, vol. 42, no. 2, pp. 291–306, 2018.

[45] A. Millane, Z. Taylor, H. Oleynikova, J. Nieto, R. Siegwart, and C. Cadena, "C-blox: A scalable and consistent tsdf-based dense mapping approach," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2018, pp. 995–1002.

[46] L. Han, F. Gao, B. Zhou, and S. Shen, "Fiesta: Fast incremental euclidean distance fields for online motion planning of aerial robots," *arXiv preprint arXiv:1903.02144*, 2019.

[47] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance transforms of sampled functions," *Theory of computing*, vol. 8, no. 1, pp. 415–428, 2012.

[48] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *arXiv preprint arXiv:1708.03852*, 2017.

[49] T. Lee, M. Leoky, and N. H. McClamroch, "Geometric tracking control of a quadrotor uav on se (3)," in *Decision and Control (CDC), 2010 49th IEEE Conference on*, 2010, pp. 5420–5425.